대학원 인공지능융합학과 I 전남대학교

Improving LLM-based code completion using LR parsing

Md Monir Ahammod Bin Atique and Kwanghoon Choi, Chonnam National University

Abstract

Code completion is essential in modern IDEs but traditional methods often overwhelm users with long, static suggestion lists. We propose a hybrid approach that combines LR parsing with large language models (LLMs) to improve accuracy and usability. LR parsing extracts structural candidates from the grammar, which are then refined into complete code suggestions by the LLM, referencing ranked examples from open-source programs. We evaluate whether LLMs benefit from these structural inputs by comparing completions with and without them. Our method is implemented in VSCode extensions for Microsoft Small Basic (SB) and C, and being language-agnostic, it can support any LR-parsable language. Results show that our integration improves both completion quality and user experience.

Introduction

Modern IDEs support code completion, but traditional methods show unranked, lengthy lists that reduce efficiency. LR parsing-based methods generate structural candidates from grammar rules, ensuring syntactic correctness introduced by Sasano and Choi [1]. They introduced a way to rank these candidates using open-source project data. However, these still require manual editing into usable code. LLMs like ChatGPT can generate full code but often lack control and structure without guidance. We propose a hybrid approach that combines ranked structural candidates from LR parsing with LLMs to generate code completions. Implemented in VSCode extensions for Small Basic and C, our system improves completion quality and is applicable to any language with an LR grammar.

Background & Methodology

Prior work [2] introduced a ranked syntax completion method using LR parsing, which extracts structural candidates from parser states and ranks them based on open-source usage. These suggestions ensure syntactic correctness, but require manual editing, limiting usability. To address this, we propose a hybrid system (Fig 1) that combines LR parsing with LLMs. The offline phase builds a ranked candidate database. The online phase retrieves top candidates and guides the LLM to generate code completions.

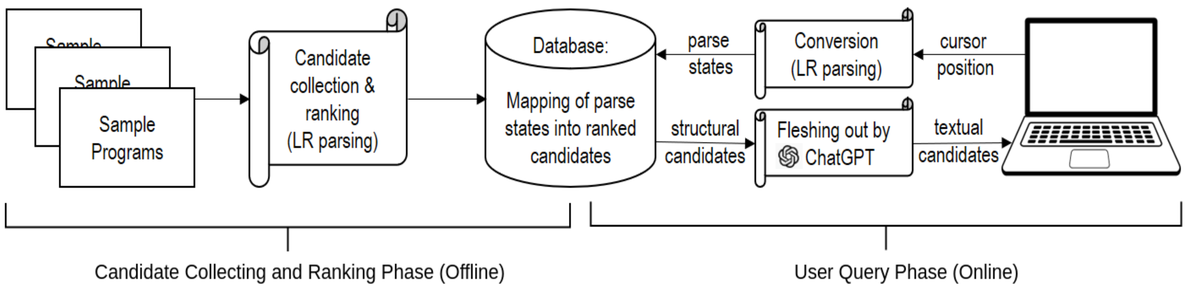


Fig 1. Overview of the system architecture.

This study explores two key questions: (RQ1) whether LR structural candidates enhance LLM-based code completion, and (RQ2) how to effectively select and use these candidates to guide the LLM for better results. This system is implemented as VSCode extensions for Small Basic and C, and is language-agnostic (applicable to any language with an LR grammar) which is depicted in Fig 2.

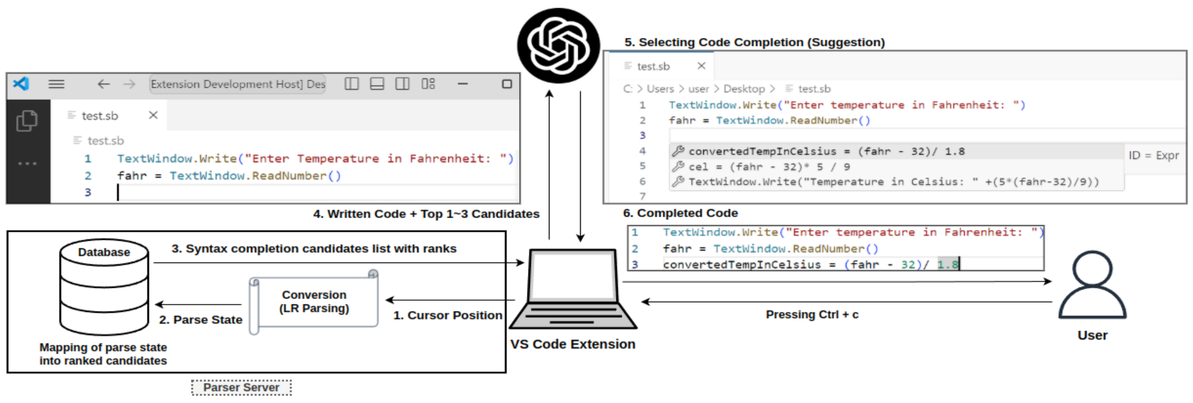


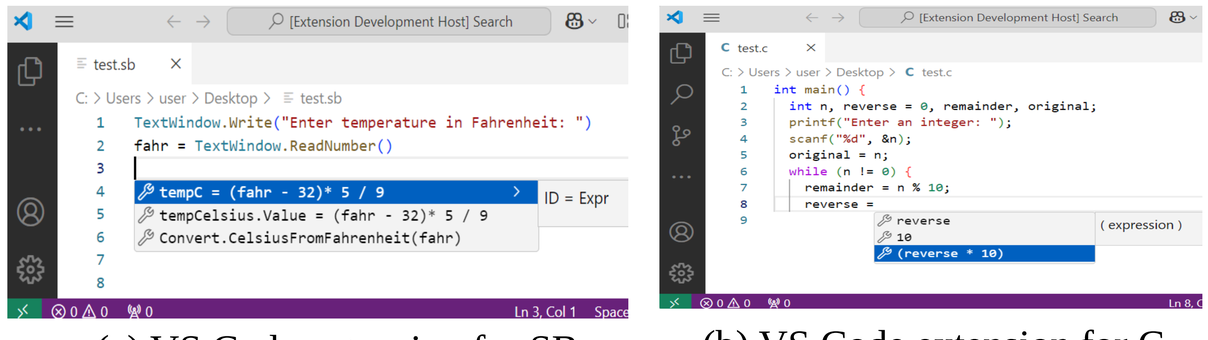
Fig 2. VSCode extension workflow diagram.

Experiment Results & VS Code Extension Implementation

We evaluated four strategies for LLM-guided code completion: WithIdealGuide (uses the correct LR structural candidate), WithinTop3Guide (selects the best among the top 3 ranked candidates), WithTop1Guide (uses only the top-ranked candidate), and WithoutGuide (no structural guidance). In Table 1, WithIdealGuide achieved the highest accuracy, confirming that LR structural candidates significantly improve LLM performance (RQ1). WithinTop3Guide closely approached ideal results, demonstrating it as a practical and effective strategy (RQ2). Our VSCode extensions for SB and C demonstrate real-time code completion, integrating structural and textual suggestions directly at the cursor position which is shown in Fig 3.

Table 1. Comparative analysis of experimental results.

PLs	Experiment types	SacreBLEU (%)	SequenceMatcher (%)
Small Basic	WithoutGuide	40.798	37.897
	WithIdealGuide	49.790	44.703
	WithinTop3Guide	45.733	43.897
	WithTop1Guide	38.524	37.097
C11	WithoutGuide	15.472	15.074
	WithIdealGuide	28.368	28.658
	WithinTop3Guide	26.222	27.810
	WithTop1Guide	20.217	20.464



(a) VS Code extension for SB (b) VS Code extension for C

Fig 3. VSCode extension examples in Small Basic and C.

Conclusions & Future Work

This work advances LLM-based code completion by combining LR parsing structural candidates with textual prompts, enabling more accurate and controllable code suggestions across languages. This hybrid method boosts accuracy and provides a scalable, multi-language framework for diverse development settings. Future work will focus on expanding datasets, improving usability, and developing a versatile VSCode extension that supports multiple programming languages.

References

[1] I. Sasano and K. Choi, "A text-based syntax completion method using LR parsing," *PEPM* 2021, pp. 32-43.

[2] K. Choi, S. Hwang, H. Moon, and I. Sasano, "Ranked Syntax Completion With LR Parsing," *ACM SAC* 2024, pp. 1242-1251

주최 | 과학기술정보통신부 Ministry of Science and ICT | 주관 | IITP 정보통신기획평가원 Institute of Information & Communications Technology Planning & Evaluation | AIGS 인공지능대학원협의회 Artificial Intelligence Graduate School Council